САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЁТ

ПО ДИСЦИПЛИНЕ «БЭК-ЭНД РАЗРАБОТКА»

по теме:

ЛР4: РАЗВЁРТЫВАНИЕ ПРИЛОЖЕНИЯ НА УДАЛЁННОМ СЕРВЕРЕ (СЕРВИС АРЕНДЫ НЕДВИЖИМОСТИ)

Проверил: Добряков Д. И.

_____

подпись, дата

Выполнил: Субботин Александр, БР1.1

_____

подпись, дата

Санкт-Петербург 2026

1 Задача

- провести предварительную настройку удалённого сервера (установка необходимых зависимостей);
- провести развёртывание приложения на удалённом сервере;
- настроить nginx в режиме обратного прокси для обеспечения доступа к приложению извне.

Разворачивается микросервисное приложение из ЛР2–3: `api-gateway`, `user-service`, `property-service`, `booking-service`, `chat-service`, PostgreSQL 16 и RabbitMQ 3.13.

Стек: Go 1.25 (Gin, gRPC + grpc-gateway), Docker Engine 29 + Docker Compose, nginx 1.22, ufw. Код — `labs/lab4-5`, скрипты — `labs/lab4-5/deploy`.

Сервер: VPS `13.143.182.131` (домен `rental.zzisst.space`), Debian 12 (ядро 6.1), 1 vCPU (AMD Ryzen 9 5950X), 1.9 ГБ RAM, 2 ГБ swap, диск 25 ГБ. На этой же ВМ уже работает VPN (панель PasarGuard на :8000, xray, OpenVPN, TimescaleDB на `127.0.0.1:5432`), что повлияло на ряд решений (раздел 2.3).

2 Ход работы

2.1 Подготовка приложения к промышленной эксплуатации

Compose-файл из ЛР3 был рассчитан на локальную разработку: все порты (включая БД и брокер) публиковались на `0.0.0.0`, пароли и JWT-секрет лежали прямо в `docker-compose.yaml`. Для сервера он переработан (Таблица 1).

Таблица 1. Изменения конфигурации развёртывания

Аспект	ЛР3	ЛР4
Сеть	опубликованы 5432, 5672, 15672, 8080–8084, 9082, 9083	внутренняя сеть `rental_backend`; наружу только `127.0.0.1:8080` (gateway), служебные — `127.0.0.1`
Секреты	в compose-файле	в `.env` (`${POSTGRES_PASSWORD:?}` — без него запуск невозможен), в git только `.env.example`
Перезапуск	нет	`restart: unless-stopped`
Healthcheck	только postgres, rabbitmq	у каждого сервиса (`wget /health`); `depends_on: condition: service_healthy`
Логи	без ограничений	ротация json-file: 3 × 10 МБ
Образы	root, golang 1.23/1.25	non-root пользователь `app`, golang 1.25, `-trimpath -ldflags="-s -w"`
RabbitMQ	guest/guest	отдельный пользователь из `.env`, данные в volume

Кроме того, при подготовке был исправлен дефект API-gateway: контекст, передаваемый в `RegisterPropertyServiceHandlerFromEndpoint`, создавался через `context.WithTimeout(…, 15*time.Second)`. grpc-gateway закрывает соединение по `ctx.Done()`, поэтому через 15 секунд после старта все gRPC-маршруты (`/properties`, `/bookings`, …) начинали отвечать ошибкой. Теперь используется `context.Background()`. Также пустые `address`/`rules` и невалидные `check_in_time` больше не приводят к ошибке вставки в БД при создании объявления.

```
x-service: &service
  restart: unless-stopped
  logging: *logging
  networks: [backend]

  api-gateway:
    <<: *service
    build: ./api-gateway
    ports:
      - "${GATEWAY_BIND:-127.0.0.1:8080}:8080"   # единственная точка входа
    healthcheck:
      test: ["CMD-SHELL", "wget -qO- http://127.0.0.1:8080/health >/dev/
null || exit 1"]
    depends_on:
      user-service:     { condition: service_healthy }
      property-service: { condition: service_healthy }
```

2.2 Предварительная настройка сервера

Настройка автоматизирована идемпотентным скриптом `deploy/scripts/bootstrap.sh`, который запускается от root:

1. установка `git`, `rsync`, `curl`, `jq`, `ufw`, `nginx`; на чистой ВМ — также `apt upgrade` и `unattended-upgrades`;
2. Docker Engine и плагин `docker compose` из официального репозитория Docker, ротация логов по умолчанию в `/etc/docker/daemon.json` (только при свежей установке Docker);
3. пользователь `deploy`: группа `docker`, SSH-ключ в `authorized_keys`, `sudo` без пароля только на `nginx -t`, `systemctl reload nginx` и установку конфига сайта (нужно для CI в ДЗ6);
4. swap-файл 4 ГБ — без него сборка пяти Go-образов на ВМ с 2 ГБ RAM падает по OOM;
5. firewall `ufw`: `deny incoming`, открыты только `OpenSSH` и `Nginx Full` (80/443), а также порт 9100 node-exporter'а — только с bridge-интерфейса `br-rental` docker-сети приложения (нужно для ЛР5);
6. по флагу `DISABLE_PASSWORD_AUTH=true` — запрет входа по паролю.

```
sudo SSH_PUBKEY="$(cat ~/.ssh/id_ed25519.pub)" bash deploy/scripts/
bootstrap.sh
```

Так как сервер не новый, скрипт сделан безопасным для уже работающей системы: если Docker уже установлен, `daemon.json` не создаётся и docker не перезапускается (иначе перезапустились бы контейнеры VPN); если `ufw` уже включён, политики по умолчанию не меняются — только добавляются правила; флаг `SKIP_UPGRADE=true` отключает `apt upgrade`. На сервере скрипт был запущен так:

```
# SKIP_UPGRADE=true SSH_PUBKEY='ssh-ed25519 AAAA… github-actions-rental-
deploy' \
    bash /opt/rental/deploy/scripts/bootstrap.sh
==> 2/7 Docker Engine + compose plugin
docker уже был установлен — daemon.json и перезапуск пропущены
==> 3/7 Пользователь deploy
Adding user `deploy' ...
==> 4/7 Swap 4G          (swap уже есть — пропущено)
==> 5/7 Firewall (ufw)
==> Готово
Docker version 29.3.0, build 5927d80
Docker Compose version v5.1.1
```

```
nginx version: nginx/1.22.1
Status: active
Default: deny (incoming), allow (outgoing), allow (routed)
To                         Action      From
22/tcp (OpenSSH)           ALLOW IN    Anywhere
80,443/tcp (Nginx Full)    ALLOW IN    Anywhere
9100/tcp on br-rental      ALLOW IN    Anywhere    # node-exporter <-
prometheus
…                          (правила VPN: 8000, 58234, 62050, 1194/udp,
51820/udp, tun0)
```

Важная деталь: порты, опубликованные Docker'ом через `ports:`, добавляются в цепочку `DOCKER` iptables и обходят правила `ufw`. Поэтому защита строится не на firewall, а на том, что в compose-файле всё публикуется только на `127.0.0.1`.

2.3 Развёртывание приложения

2.3.1 Особенности общей ВМ

На сервере свободно около 600 МБ RAM, поэтому:

- **образы не собираются на сервере**: компиляция пяти Go-сервисов на 1 vCPU ушла бы в swap и замедлила VPN. Dockerfile'ы переведены на кросс-компиляцию (`FROM --platform=$BUILDPLATFORM`, `GOOS=$TARGETOS GOARCH=$TARGETARCH`), образы собираются на рабочей машине под `linux/amd64` без эмуляции и передаются по SSH скриптом `push-images.sh` (`docker save | ssh docker load`, 13 МБ на образ). В ДЗ6 их собирает GitHub Actions и публикует в GHCR;
- **лимиты памяти** (`mem_limit`: 128 МБ на Go-сервис, 64–512 МБ на компоненты мониторинга) и `oom_score_adj: 300` у всех контейнеров приложения: при нехватке памяти ядро в первую очередь завершит их, а не процессы VPN;
- порт `5432` занят TimescaleDB панели, поэтому Postgres приложения опубликован на `127.0.0.1:5433` (`POSTGRES_BIND`);
- Docker не перезапускался, системные пакеты не обновлялись — установлены только `nginx` и `rsync`.

2.3.2 Деплой

Проект копируется на сервер в `/opt/rental` (вручную — `rsync`, в ДЗ6 — из GitHub Actions). `.env` создаётся из шаблона на самом сервере, секреты генерируются там же (`openssl rand`), чтобы не передавать их по сети:

```
rsync -az --delete --exclude .env --exclude docs/ ./ root@13.143.182.131:/
opt/rental/
ssh root@13.143.182.131 'cd /opt/rental && cp .env.example .env && \
  for k in POSTGRES_PASSWORD RABBITMQ_PASSWORD GRAFANA_ADMIN_PASSWORD; do \
    sed -i "s|^$k=.*|$k=$(openssl rand -hex 16)|" .env; done && \
  sed -i "s|^JWT_SECRET=.*|JWT_SECRET=$(openssl rand -hex 32)|" .env'
./deploy/scripts/push-images.sh root@13.143.182.131 linux/amd64
ssh root@13.143.182.131 'cd /opt/rental && DEPLOY_MODE=local
IMAGE_TAG=lab4-202611041329 \
  ./deploy/scripts/deploy.sh'
```

Скрипт `deploy.sh` поддерживает три режима (`DEPLOY_MODE`): `build` — собрать образы на сервере, `pull` — скачать из реестра (CI), `local` — образы уже загружены. Он записывает в `.env` версию образов и релиза (`IMAGE_TAG`, `SENTRY_RELEASE`), скачивает внешние образы, выполняет `docker compose up -d --no-build --remove-orphans`, ждёт, пока у всех контейнеров с healthcheck'ом статус станет `healthy` (при таймауте выводит их логи и завершается с ошибкой), проверяет `/health` gateway и удаляет неиспользуемые образы. Вывод на сервере:

```
==> mode=local image=rental-*:lab4-202611041329 release=lab4-202611041329
==> запуск
==> ожидание healthcheck'ов
NAME                      IMAGE                                                 
STATUS
mon_grafana               grafana/grafana:13.0.12                               Up 19
minutes
mon_prometheus            prom/prometheus:v3.13.5                               Up 19
minutes
…
rental_api_gateway        rental-api-gateway:lab4-202611041329                  Up 17
minutes (healthy)
rental_booking_service    rental-booking-service:lab4-202611041329              Up 17
minutes (healthy)
rental_broker             rabbitmq:3.13-management-alpine                       Up 19
minutes (healthy)
rental_chat_service       rental-chat-service:lab4-202611041329                 Up 18
minutes (healthy)
rental_db                 postgres:16-alpine                                    Up 19
```

```
minutes (healthy)
rental_property_service   rental-property-service:lab4-202611041329   Up 17
minutes (healthy)
rental_user_service       rental-user-service:lab4-202611041329       Up 18
minutes (healthy)
{"service":"api-gateway","status":"ok"}
==> deploy OK
```

Потребление памяти приложением: Go-сервисы — 8–13 МБ каждый, PostgreSQL — 27 МБ, RabbitMQ — 46 МБ.

2.4 Настройка nginx как обратного прокси

Конфиг собран из нескольких файлов в `deploy/nginx/`, которые устанавливает скрипт `setup-nginx.sh <домен> [доп. имена]`. Он подставляет `server_name`, выбирает вариант (HTTP или HTTPS — если для домена уже выпущен сертификат), проверяет конфиг (`nginx -t`) и перезагружает nginx:

- `snippets/rental-http.conf` — общее для http-контекста: `upstream`'ы и зона rate limit;
- `snippets/rental-locations.conf` — общая часть server-блока: заголовки, проксирование, `location`'ы;
- `rental.conf` / `rental-tls.conf` — сам сайт (HTTP либо HTTPS + редирект);
- `default.conf` — сервер по умолчанию для запросов с чужими именами.

```
# snippets/rental-http.conf
limit_req_zone $binary_remote_addr zone=auth:10m rate=10r/s;
upstream api_gateway { server 127.0.0.1:8080; keepalive 32; }
upstream grafana     { server 127.0.0.1:3000; keepalive 8; }

# snippets/rental-locations.conf
client_max_body_size 10m;
server_tokens off;
add_header X-Content-Type-Options nosniff always;
add_header X-Frame-Options SAMEORIGIN always;
proxy_http_version 1.1;
proxy_set_header Host              $host;
proxy_set_header X-Real-IP         $remote_addr;
proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header Connection        "";
```

```
location /api/v1/auth/ {            # защита от перебора паролей
    limit_req zone=auth burst=20 nodelay;
    limit_req_status 429;
    proxy_pass http://api_gateway;
}
location /         { proxy_pass http://api_gateway; }   # API + Swagger
location /grafana/ { proxy_pass http://grafana; }       # ЛР5
location /debug/   { return 404; }                      # тестовый маршрут
Sentry
```

Особенности конфига:

- `upstream` с `keepalive` и `Connection ""` — соединения с gateway переиспользуются;
- `X-Forwarded-*` — сервисы и логи видят реальный IP клиента;
- для `/grafana/api/live/` (websocket) заголовки `Upgrade`/`Connection` задаются отдельно; так как `proxy_set_header` внутри `location` отменяет наследование, остальные заголовки там повторены;
- rate limit на эндпоинты аутентификации: 10 запросов/с с IP, всплеск до 20, далее `429`.

Снаружи слушает только nginx, всё остальное — на `127.0.0.1` (`ss -tlnp` на сервере, порты приложения):

```
LISTEN 0.0.0.0:80             nginx
LISTEN 0.0.0.0:443            nginx
LISTEN 127.0.0.1:8080         docker-proxy    # api-gateway
LISTEN 127.0.0.1:3000         docker-proxy    # Grafana
LISTEN 127.0.0.1:5433         docker-proxy    # PostgreSQL (SSH-туннель)
LISTEN 127.0.0.1:15672        docker-proxy    # RabbitMQ management (SSH-
туннель)
LISTEN 127.0.0.1:8025         docker-proxy    # Mailpit (SSH-туннель)
LISTEN *:9100                 node_exporter   # закрыт ufw, кроме br-rental
```

Проверка работы прокси с внешней машины (40 параллельных запросов логина — часть прошла, остальные отсечены лимитом):

```
$ for u in /health /swagger/index.html /api/v1/properties /grafana/api/
health /debug/sentry; do
    curl -s -o /dev/null -w "$u -> %{http_code}\n" http://13.143.182.131$u;
done
/health -> 200
/swagger/index.html -> 200
/api/v1/properties -> 200
/grafana/api/health -> 200
/debug/sentry -> 404
```

```
$ curl -sI http://13.143.182.131/health | grep -E 'Server|X-Frame'
Server: nginx
X-Frame-Options: SAMEORIGIN

$ for i in $(seq 1 40); do curl -s -o /dev/null -w '%{http_code}\n' \
    -X POST http://13.143.182.131/api/v1/auth/login -d '{}' & done | sort |
uniq -c
  26 400
  14 429
```

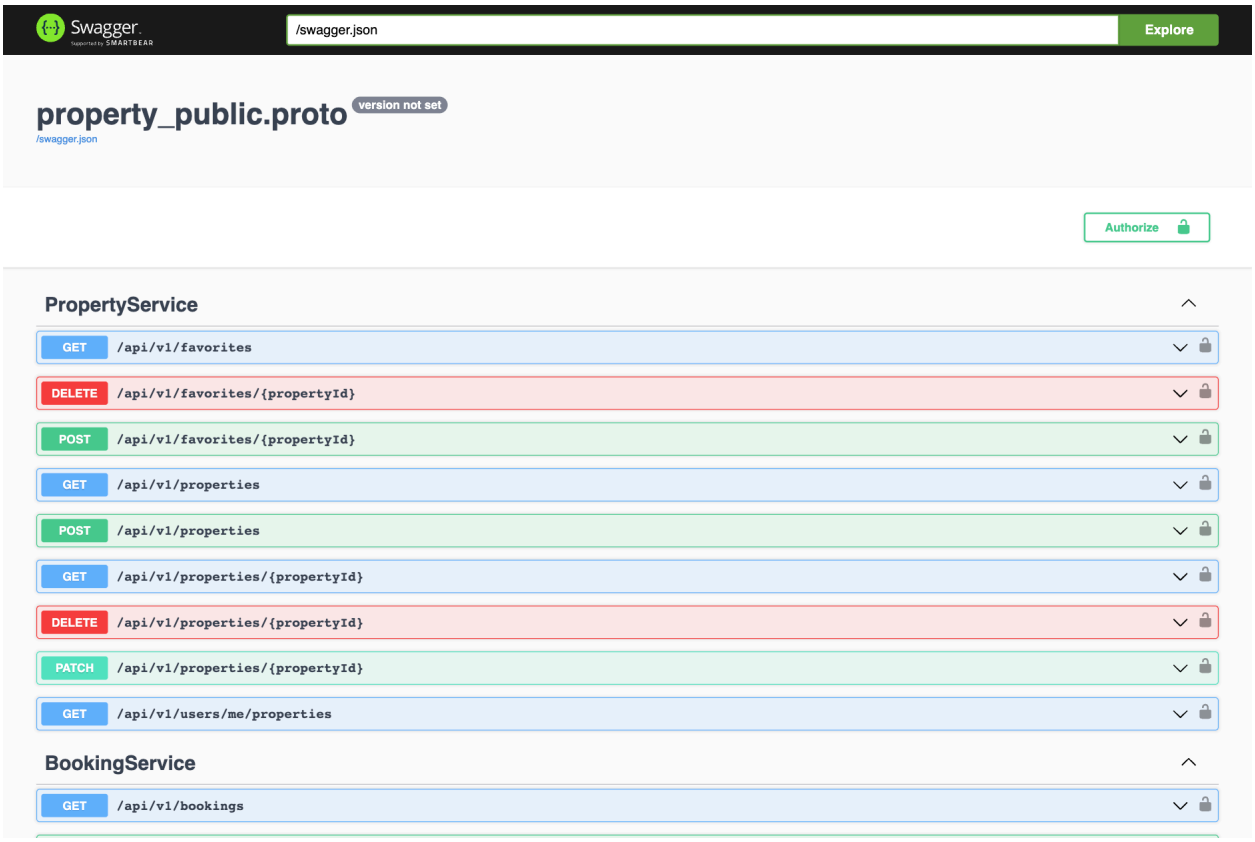


Рисунок 1. Swagger UI API-gateway по адресу http://13.143.182.131/swagger/index.html

2.5 Домен и HTTPS

Для сервиса добавлен поддомен `rental.zzisst.space` (A-запись на `13.143.182.131` в DNS-зоне домена у регистратора). Корень `zzisst.space` оставлен свободным, панель VPN по-прежнему доступна по своему поддомену на порту 8000.

Сертификат выпущен Let’s Encrypt тем же скриптом:

```
CERTBOT=1 ./deploy/scripts/setup-nginx.sh rental.zzisst.space
13.143.182.131
```

Скрипт сначала ставит HTTP-вариант сайта (чтобы прошла проверка владения доменом http-01), затем выполняет `certbot certonly --nginx -d rental.zzisst.space --deploy-hook "systemctl reload nginx"` и переключает сайт на `rental-tls.conf`. Конфиг nginx остаётся под контролем репозитория: certbot только получает сертификат и не редактирует файлы сайта.

```
server {                        # http → https для домена
    listen 80;
    server_name rental.zzisst.space;
    location / { return 301 https://$host$request_uri; }
}
server {
    listen 443 ssl http2;
    server_name rental.zzisst.space;
    ssl_certificate     /etc/letsencrypt/live/rental.zzisst.space/
fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/rental.zzisst.space/
privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    add_header Strict-Transport-Security "max-age=31536000" always;
    include /etc/nginx/snippets/rental-locations.conf;
}
server {                        # доступ по IP без TLS — для проверок
    listen 80;
    server_name 13.143.182.131;
    include /etc/nginx/snippets/rental-locations.conf;
}

# default.conf — всё остальное (другие домены на этом IP, сканеры)
server { listen 80 default_server;  server_name _; return 444; }
server { listen 443 ssl default_server; server_name _; ssl_reject_handshake
on; }
```

Сервер по умолчанию понадобился потому, что на тот же IP указывает и поддомен панели VPN: без него `http://panel.zzisst.space` открывал бы приложение аренды. Теперь такие запросы сбрасываются (444), а TLS-рукопожатие с чужим SNI отклоняется.

Проверка:

```
$ curl -sI http://rental.zzisst.space/swagger/index.html | grep -E '^HTTP|
^Location'
HTTP/1.1 301 Moved Permanently
Location: https://rental.zzisst.space/swagger/index.html
```

```
$ for u in /health /swagger/index.html /api/v1/properties /grafana/api/
health /debug/sentry; do
    curl -s -o /dev/null -w "$u -> %{http_code}\n" https://rental.zzisst.
space$u; done
/health -> 200
/swagger/index.html -> 200
/api/v1/properties -> 200
/grafana/api/health -> 200
/debug/sentry -> 404

$ echo | openssl s_client -connect rental.zzisst.space:443 -servername
rental.zzisst.space \
    | openssl x509 -noout -subject -issuer -dates
subject=CN=rental.zzisst.space
issuer=C=US, O=Let's Encrypt, CN=YE1
notBefore=Nov  4 11:49:07 2026 GMT
notAfter=Feb  2 11:49:06 2027 GMT

$ curl -s -o /dev/null -w '%{http_code}' http://panel.zzisst.space/   #
чужой Host
000          # соединение сброшено (444)

$ certbot renew --dry-run
Simulating renewal of an existing certificate for rental.zzisst.space
  /etc/letsencrypt/live/rental.zzisst.space/fullchain.pem (success)
```

Сертификат продлевается автоматически таймером `certbot.timer` (дважды в сутки), после продления nginx перезагружается deploy-hook'ом. Адрес Grafana в `.env` (`GRAFANA_ROOT_URL=https://rental.zzisst.space/grafana/`) и переменная `PUBLIC_URL` для smoke-теста в CI (ДЗ6) переведены на домен.

2.6 Проверка основного сценария через nginx

```
$ curl -X POST http://13.143.182.131/api/v1/auth/register \
    -d
'{"email":"demo@rental.local","password":"demo12345","full_name":"Demo
User"}'
{"id":"4b5f6006-…","email":"demo@rental.local","full_name":"Demo
User","is_active":true,…}

$ TOKEN=$(curl -s -X POST http://13.143.182.131/api/v1/auth/login \
    -d '{"email":"demo@rental.local","password":"demo12345"}' | jq -
r .access_token)

$ curl -X POST http://13.143.182.131/api/v1/properties -H "Authorization:
```

```
Bearer $TOKEN" \
    -d '{"title":"Квартира у Невского","property_type":"apartment",
         "price_per_night":4500,"city":"Санкт-
Петербург","max_guests":3,"check_in_time":"14:00"}'
{"id":"37fb7e7c-…","title":"Квартира у
Невского","price_per_night":4500,"currency":"RUB",…}

$ curl "http://13.143.182.131/api/v1/properties?city=Санкт-Петербург"
{"items":[{"id":"37fb7e7c-…","title":"Квартира у
Невского",…}],"pagination":{"limit":20}}
```

Запросы проходят цепочку nginx → api-gateway → (HTTP) user-service / (gRPC) property-service → PostgreSQL.

3 Вывод

Приложение подготовлено к работе на сервере: секреты вынесены в `.env`, все сервисы закрыты во внутренней docker-сети, у каждого есть healthcheck и политика перезапуска. Настройка сервера (Docker, nginx, firewall, swap, пользователь для деплоя) и развёртывание автоматизированы скриптами `bootstrap.sh` и `deploy.sh`, что используется в ДЗ6 для автодеплоя. Развёртывание выполнено на общей ВМ с работающим VPN без его остановки: образы собираются вне сервера, а лимиты памяти и приоритет OOM защищают соседние сервисы. Внешний доступ организован через nginx в режиме обратного прокси по домену `rental.zzisst.space` с HTTPS (Let’s Encrypt, автопродление), ограничением частоты запросов к аутентификации и проксированием Grafana для ЛР5. Попутно исправлен дефект gateway, из-за которого gRPC-маршруты переставали работать через 15 секунд после старта.